

1. Write a program to implement removal of recursion for – Finding maximum from array.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

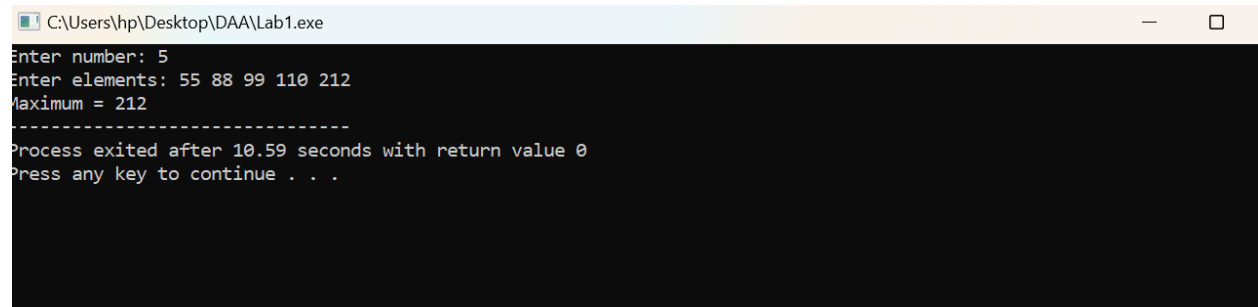
    int max = arr[0];

    for (int i = 1; i < n; i++)
        if (arr[i] > max)
            max = arr[i];

    cout << "Maximum = " << max;

    return 0;
}
```

Output:



```
C:\Users\hp\Desktop\DAA\Lab1.exe
Enter number: 5
Enter elements: 55 88 99 110 212
Maximum = 212
-----
Process exited after 10.59 seconds with return value 0
Press any key to continue . . .
```

2. Write a program to implement removal of recursion for – Binomial Coefficient $B(n,m) = B(n-1, m-1) + B(n-1,m)$, $B(n,n) = B(n,0) = 1$

```
#include <iostream>

using namespace std;

int main() {
    int n, m;

    cout << "Enter n and m: ";

    cin >> n >> m;

    if (m < 0 || m > n) {
        cout << "Invalid input";

        return 0;}

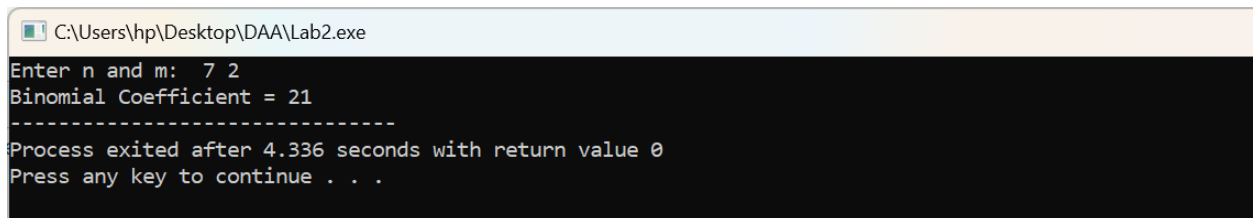
    int C[n + 1][m + 1];

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m && j <= i; j++) {
            if (j == 0 || j == i)
                C[i][j] = 1;
            else
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j];
        }
    }

    cout << "Binomial Coefficient = " << C[n][m];

    return 0; }
```

Output:



```
C:\Users\hp\Desktop\DAA\Lab2.exe
Enter n and m: 7 2
Binomial Coefficient = 21
-----
Process exited after 4.336 seconds with return value 0
Press any key to continue . . .
```

3. Write a program to implement removal of recursion for – Searching element from array.

```
#include <iostream>

using namespace std;

int main() {
    int n, key;

    cout << "Enter number of elements: ";

    cin >> n;

    int arr[n];

    cout << "Enter array elements: ";

    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter element to search: ";

    cin >> key;

    int pos = -1;

    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            pos = i;
            break;
        }
    }

    if (pos != -1)
        cout << "Element found at position " << pos + 1;

    else
        cout << "Element not found";

    return 0;
}
```

Output:

 C:\Users\hp\Desktop\DAA\Lab3.exe

Enter number of elements: 5

Enter array elements: 55 66 77 88 99

Enter element to search: 66

Element found at position 2

Process exited after 10.93 seconds with return val

Press any key to continue . . . 

4. Write a program for creating Max/Min. heap using INSERT.

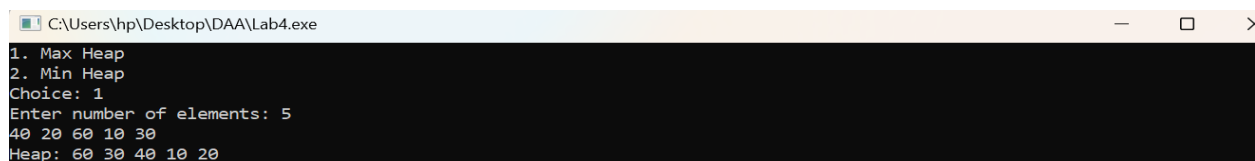
```
#include <iostream>

using namespace std;

void insert(int h[], int &n, int x, bool maxHeap) {
    h[++n] = x;
    int i = n;
    while (i > 1 && (maxHeap ? h[i] > h[i/2] : h[i] < h[i/2])) {
        swap(h[i], h[i/2]);
        i /= 2;
    }
}

int main() {
    int h[100], n = 0, x, choice, size;
    cout << "1. Max Heap\n2. Min Heap\nChoice: ";
    cin >> choice;
    cout << "Enter number of elements: ";
    cin >> size;
    for (int i = 0; i < size; i++) {
        cin >> x;
        insert(h, n, x, choice == 1);}
    cout << "Heap: ";
    for (int i = 1; i <= n; i++)
        cout << h[i] << " ";
    return 0; }
```

Output:



```
C:\Users\hp\Desktop\DAALab4.exe
1. Max Heap
2. Min Heap
Choice: 1
Enter number of elements: 5
40 20 60 10 30
Heap: 60 30 40 10 20
```

5. Write a program for creating Max/Min. heap using ADJUST/HEAPIFY.

```
#include <iostream>

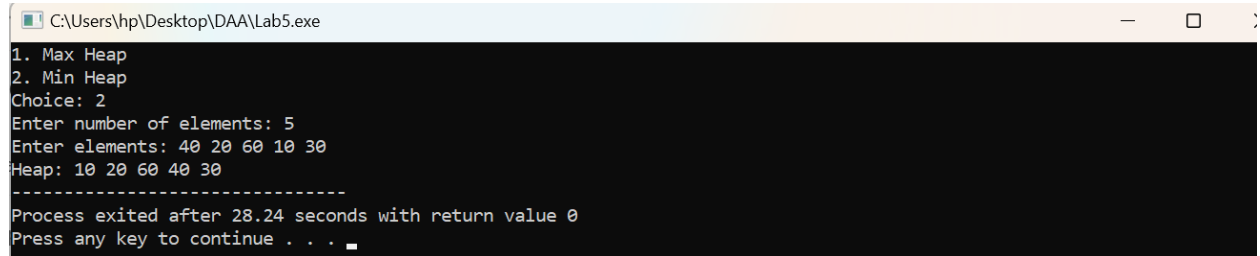
using namespace std;

void heapify(int h[], int n, int i, int max) {
    int x = i, l = 2*i, r = 2*i+1;
    if (l <= n && (max ? h[l] > h[x] : h[l] < h[x]))
        x = l;
    if (r <= n && (max ? h[r] > h[x] : h[r] < h[x]))
        x = r;
    if (x != i) {
        swap(h[i], h[x]);
        heapify(h, n, x, max);
    }
}

int main() {
    int h[100], n, choice;
    cout << "1. Max Heap\n2. Min Heap\nChoice: ";
    cin >> choice;
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (int i = 1; i <= n; i++)
        cin >> h[i];
    for (int i = n/2; i >= 1; i--)
        heapify(h, n, i, choice == 1);
    cout << "Heap: ";
    for (int i = 1; i <= n; i++)
        cout << h[i] << " ";
```

```
return 0;  
  
}
```

Output:



```
C:\Users\hp\Desktop\DAA\Lab5.exe  
1. Max Heap  
2. Min Heap  
Choice: 2  
Enter number of elements: 5  
Enter elements: 40 20 60 10 30  
Heap: 10 20 60 40 30  
-----  
Process exited after 28.24 seconds with return value 0  
Press any key to continue . . .
```

6. Write a program for sorting given array in ascending/descending order with n=1000,2000,3000. Find exact time of execution using Heap Sort.

```
#include <iostream>

#include <chrono>

using namespace std;

using namespace chrono;

void heapify(int a[], int n, int i, bool asc) {
    int x = i, l = 2*i+1, r = 2*i+2;

    if (l < n && (asc ? a[l] > a[x] : a[l] < a[x]))
        x = l;

    if (r < n && (asc ? a[r] > a[x] : a[r] < a[x]))
        x = r;

    if (x != i) {
        swap(a[i], a[x]);
        heapify(a, n, x, asc);
    }
}

void heapSort(int a[], int n, bool asc) {
    for (int i = n/2-1; i >= 0; i--)
        heapify(a, n, i, asc);

    for (int i = n-1; i > 0; i--) {
        swap(a[0], a[i]);
        heapify(a, i, 0, asc);
    }
}

void test(int n, bool asc) {
    int *a = new int[n];

    for (int i = 0; i < n; i++)
        a[i] = n - i;
```

```

auto start = high_resolution_clock::now();

heapSort(a, n, asc);

auto end = high_resolution_clock::now();

cout << "n = " << n << " | Time = "

    << duration_cast<microseconds>(end-start).count()

    << " microseconds\n";

delete[] a;
}

int main() {

    int choice;

    cout << "1. Ascending\n2. Descending\nChoice: ";

    cin >> choice;

    bool asc = (choice == 1);

    cout << "\nHeap Sort Execution Time\n";

    test(1000, asc);

    test(2000, asc);

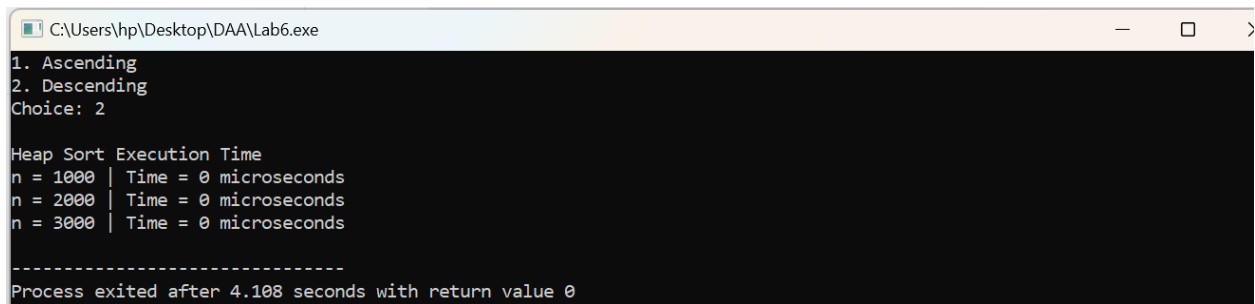
    test(3000, asc);

    return 0;

}

```

Output:



```

C:\Users\hp\Desktop\DAA\Lab6.exe
1. Ascending
2. Descending
Choice: 2

Heap Sort Execution Time
n = 1000 | Time = 0 microseconds
n = 2000 | Time = 0 microseconds
n = 3000 | Time = 0 microseconds
-----
Process exited after 4.108 seconds with return value 0

```

7. Write a program to implement Weighted UNION and Collapsing FIND operations.

```
#include <iostream>

using namespace std;

int parent[100], sizeSet[100];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]); // Collapsing Find
    return parent[x];
}

void weightedUnion(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return;
    if (sizeSet[a] < sizeSet[b]) {
        parent[a] = b;
        sizeSet[b] += sizeSet[a];
    } else {
        parent[b] = a;
        sizeSet[a] += sizeSet[b];
    }
}

int main() {
    int n, ch, a, b;
    cout << "Enter number of elements: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        parent[i] = i;
        sizeSet[i] = 1;
    }
    do {
```

```

        cout << "\n1. Weighted Union";

        cout << "\n2. Collapsing Find";

        cout << "\n3. Display";

        cout << "\n4. Exit";

        cout << "\nChoice: ";

        cin >> ch;

    if (ch == 1) {

        cin >> a >> b;

        weightedUnion(a, b);

        cout << "Union performed";

    } else if (ch == 2) {

        cin >> a;

        cout << "Root = " << find(a);

    } else if (ch == 3) {

        for (int i = 0; i < n; i++)

            cout << parent[i] << " ";

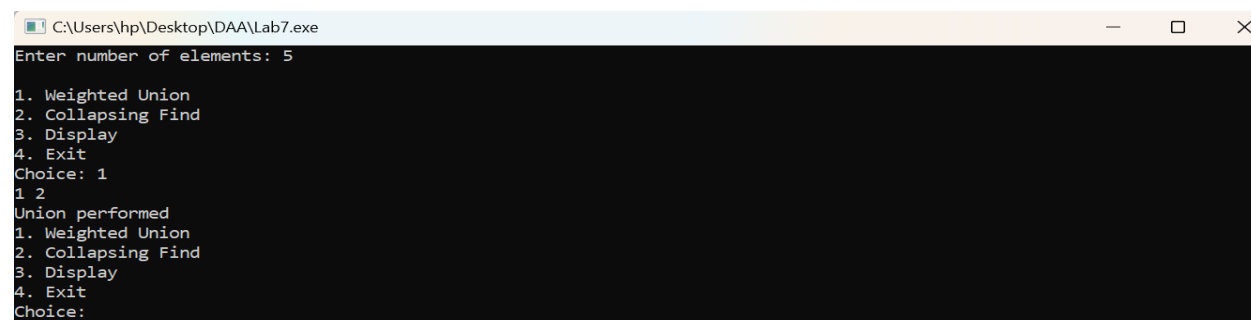
    } } while (ch != 4);

    return 0;

}

```

Output:



```

C:\Users\hp\Desktop\DAA\Lab7.exe
Enter number of elements: 5
1. Weighted Union
2. Collapsing Find
3. Display
4. Exit
Choice: 1
1 2
Union performed
1. Weighted Union
2. Collapsing Find
3. Display
4. Exit
Choice:

```

8. Write a program for searching element from given array using binary search for n=1000,2000,3000. Find exact time of execution.

```
#include <iostream>

#include <chrono>

using namespace std;

using namespace chrono;

int binarySearch(int a[], int n, int key) {
    int l = 0, h = n - 1;

    while (l <= h) {
        int m = (l + h) / 2;

        if (a[m] == key)
            return m;

        if (a[m] < key)
            l = m + 1;
        else
            h = m - 1;
    } return -1;
}

void test(int n, int key) {
    int *a = new int[n];

    for (int i = 0; i < n; i++)
        a[i] = i + 1;

    auto start = high_resolution_clock::now();

    int result = binarySearch(a, n, key);

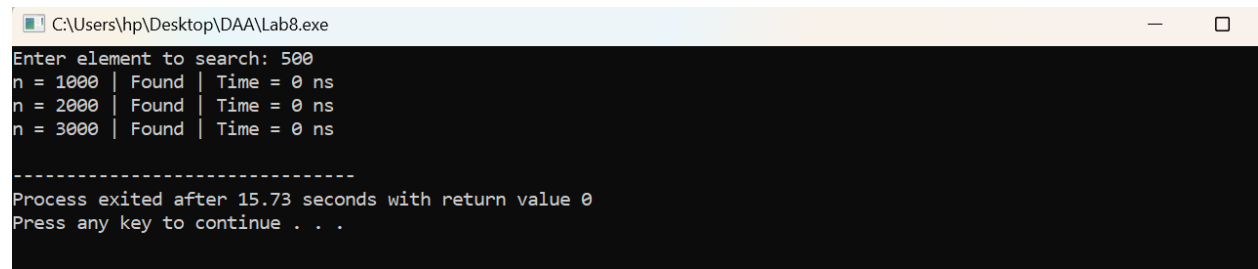
    auto end = high_resolution_clock::now();

    cout << "n = " << n
        << " | " << (result != -1 ? "Found" : "Not Found")
        << " | Time = "
```

```
        << duration_cast<nanoseconds>(end - start).count()
        << " ns\n";
delete[] a;
}

int main() {
    int key;
    cout << "Enter element to search: ";
    cin >> key;
    test(1000, key);
    test(2000, key);
    test(3000, key);
    return 0;
}
```

Output:



```
C:\Users\hp\Desktop\DAA\Lab8.exe
Enter element to search: 500
n = 1000 | Found | Time = 0 ns
n = 2000 | Found | Time = 0 ns
n = 3000 | Found | Time = 0 ns

-----
Process exited after 15.73 seconds with return value 0
Press any key to continue . . .
```

9. Write a program to find minimum and maximum from a given array using MAXMIN.

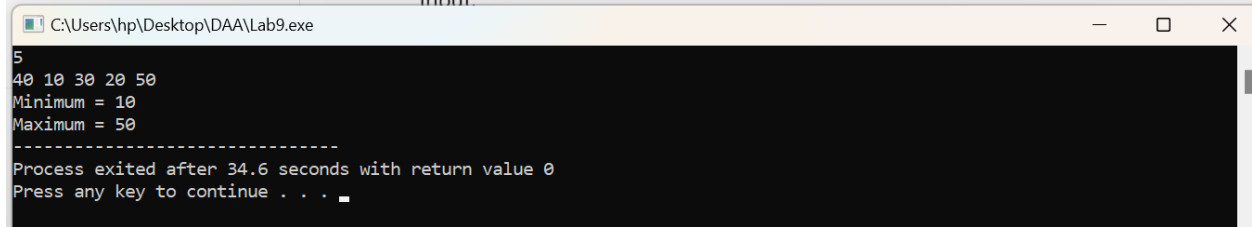
```
#include <iostream>

using namespace std;

void maxmin(int a[], int l, int h, int &mn, int &mx) {
    if (l == h)
        mn = mx = a[l];
    else {
        int m = (l + h) / 2;
        int mn1, mx1;
        maxmin(a, l, m, mn, mx);
        maxmin(a, m + 1, h, mn1, mx1);
        if (mn1 < mn) mn = mn1;
        if (mx1 > mx) mx = mx1;
    }
}

int main() {
    int n;
    cin >> n;
    int a[n];
    for (int i = 0; i < n; i++)
        cin >> a[i];
    int mn, mx;
    maxmin(a, 0, n - 1, mn, mx);
    cout << "Minimum = " << mn << endl;
    cout << "Maximum = " << mx;
    return 0;
}
```

Output:

A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\hp\Desktop\DAA\Lab9.exe". The window has standard minimize, maximize, and close buttons. The command prompt area has a black background with white text. The output shows a series of numbers, the minimum and maximum values, a separator line, and a message about the process exit time and a prompt to press a key to continue.

```
C:\Users\hp\Desktop\DAA\Lab9.exe
5
40 10 30 20 50
Minimum = 10
Maximum = 50
-----
Process exited after 34.6 seconds with return value 0
Press any key to continue . . .
```

10. Write a program for sorting given array in ascending/descending order with n=1000,2000,3000 find exact time of execution using a) Merge Sort, b) Quick Sort.

```
#include <iostream>

#include <chrono>

using namespace std;

using namespace chrono;

void merge(int a[], int l, int m, int h, bool asc) {
    int i = l, j = m + 1, k = 0;
    int t[h - l + 1];
    while (i <= m && j <= h)
        t[k++] = (asc ? a[i] <= a[j] : a[i] >= a[j]) ? a[i++] : a[j++];
    while (i <= m) t[k++] = a[i++];
    while (j <= h) t[k++] = a[j++];
    for (i = l, k = 0; i <= h; i++, k++)
        a[i] = t[k];
}

void mergeSort(int a[], int l, int h, bool asc) {
    if (l < h) {
        int m = (l + h) / 2;
        mergeSort(a, l, m, asc);
        mergeSort(a, m + 1, h, asc);
        merge(a, l, m, h, asc);
    }
}

void quickSort(int a[], int l, int h, bool asc) {
    if (l >= h) return;
    int p = a[h], i = l - 1;
```

```

for (int j = l; j < h; j++)
    if (asc ? a[j] <= p : a[j] >= p)
        swap(a[++i], a[j]);
swap(a[i + 1], a[h]);
int x = i + 1;
quickSort(a, l, x - 1, asc);
    quickSort(a, x + 1, h, asc);
}

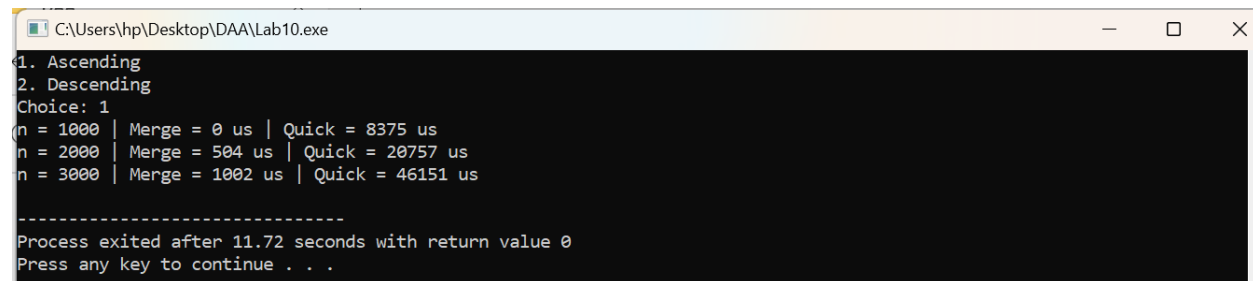
void test(int n, bool asc) {
    int *a = new int[n], *b = new int[n];
    for (int i = 0; i < n; i++)
        a[i] = b[i] = n - i;
    auto s = high_resolution_clock::now();
    mergeSort(a, 0, n - 1, asc);
    auto e = high_resolution_clock::now();
    auto mt = duration_cast<microseconds>(e - s).count();
    s = high_resolution_clock::now();
    quickSort(b, 0, n - 1, asc);
    e = high_resolution_clock::now();
    auto qt = duration_cast<microseconds>(e - s).count();
    cout << "n = " << n
        << " | Merge = " << mt << " us"
        << " | Quick = " << qt << " us\n";
    delete[] a;
    delete[] b;
}

int main() {
    int choice;

```

```
    cout << "1. Ascending\n2. Descending\nChoice: ";  
  
    cin >> choice;  
  
    bool asc = choice == 1;  
  
    test(1000, asc);  
  
    test(2000, asc);  
  
    test(3000, asc);  
  
    return 0;  
}
```

Output:



```
C:\Users\hp\Desktop\DAA\Lab10.exe  
1. Ascending  
2. Descending  
Choice: 1  
n = 1000 | Merge = 0 us | Quick = 8375 us  
n = 2000 | Merge = 504 us | Quick = 20757 us  
n = 3000 | Merge = 1002 us | Quick = 46151 us  
  
-----  
Process exited after 11.72 seconds with return value 0  
Press any key to continue . . .
```

11. Write a program for matrix multiplication using Strassen's Matrix Multiplication.

```
#include <iostream>

using namespace std;

int main() {
    int A[2][2], B[2][2], C[2][2];

    int p1, p2, p3, p4, p5, p6, p7;

    cout << "Enter first matrix:\n";

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            cin >> A[i][j];

    cout << "Enter second matrix:\n";

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            cin >> B[i][j];

    p1 = A[0][0] * (B[0][1] - B[1][1]);
    p2 = (A[0][0] + A[0][1]) * B[1][1];
    p3 = (A[1][0] + A[1][1]) * B[0][0];
    p4 = A[1][1] * (B[1][0] - B[0][0]);
    p5 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
    p6 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
    p7 = (A[0][0] - A[1][0]) * (B[0][0] + B[0][1]);

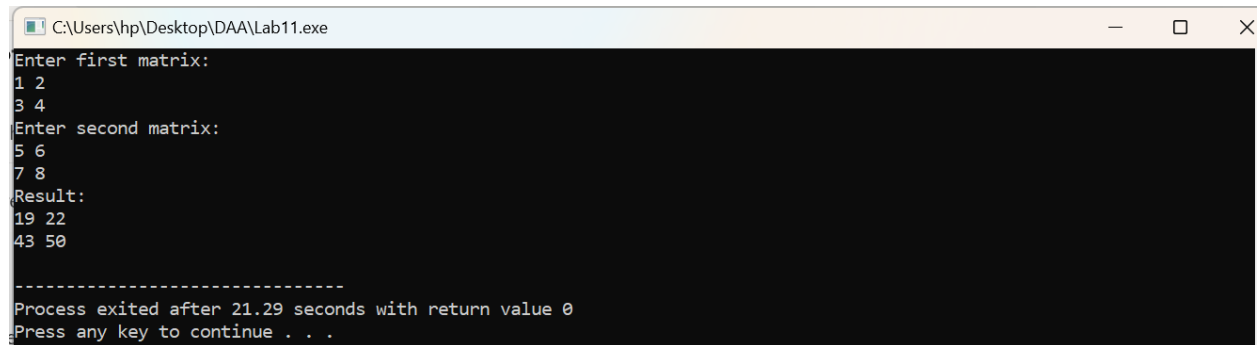
    C[0][0] = p5 + p4 - p2 + p6;
    C[0][1] = p1 + p2;
    C[1][0] = p3 + p4;
    C[1][1] = p5 + p1 - p3 - p7;
```

```
    cout << "Result:\n";

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++)
            cout << C[i][j] << " ";
        cout << endl;
    }

    return 0;
}
```

Output:



```
C:\Users\hp\Desktop\DAA\Lab11.exe
Enter first matrix:
1 2
3 4
Enter second matrix:
5 6
7 8
Result:
19 22
43 50

-----
Process exited after 21.29 seconds with return value 0
Press any key to continue . . .
```

12. Write a program to find solution of Fractional Knapsack instance.

```
#include <iostream>

#include <algorithm>

using namespace std;

struct Item {
    int weight, profit;
    double ratio;
};

bool compare(Item a, Item b) {
    return a.ratio > b.ratio;
}

int main() {
    int n, capacity;
    double profit = 0;

    cout << "Enter number of items: ";
    cin >> n;

    Item a[n];

    cout << "Enter weight and profit:\n";
    for (int i = 0; i < n; i++) {
        cin >> a[i].weight >> a[i].profit;
        a[i].ratio = (double)a[i].profit / a[i].weight;
    }

    cout << "Enter capacity: ";
    cin >> capacity;

    sort(a, a + n, compare);
```

```

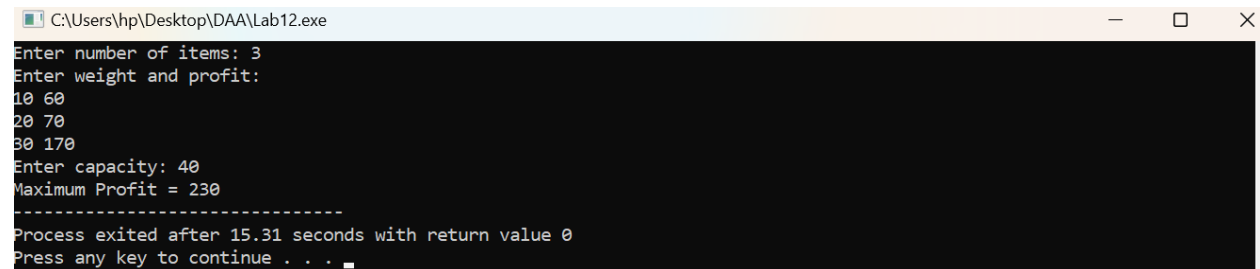
for (int i = 0; i < n && capacity > 0; i++) {
    if (a[i].weight <= capacity) {
        capacity -= a[i].weight;
        profit += a[i].profit;
    } else {
        profit += a[i].ratio * capacity;
        capacity = 0;
    }
}

cout << "Maximum Profit = " << profit;

return 0;
}

```

Output:



```

C:\Users\hp\Desktop\DAA\Lab12.exe
Enter number of items: 3
Enter weight and profit:
10 60
20 70
30 170
Enter capacity: 40
Maximum Profit = 230
-----
Process exited after 15.31 seconds with return value 0
Press any key to continue . . .

```

13. Write a program to find Minimum Spanning Tree using Prim's algorithm.

```
#include <iostream>

using namespace std;

#define INF 9999

int main() {
    int n;

    cout << "Enter number of vertices: ";

    cin >> n;

    int g[n][n], key[n], parent[n], used[n];

    cout << "Enter adjacency matrix:\n";

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            cin >> g[i][j];
            if (g[i][j] == 0)
                g[i][j] = INF;
        }

    for (int i = 0; i < n; i++) {
        key[i] = INF;
        parent[i] = -1;
        used[i] = 0;
    }

    key[0] = 0;

    int cost = 0;

    cout << "\nEdges in MST:\n";

    for (int c = 0; c < n; c++) {
        int u = -1;

        for (int i = 0; i < n; i++)
```

```

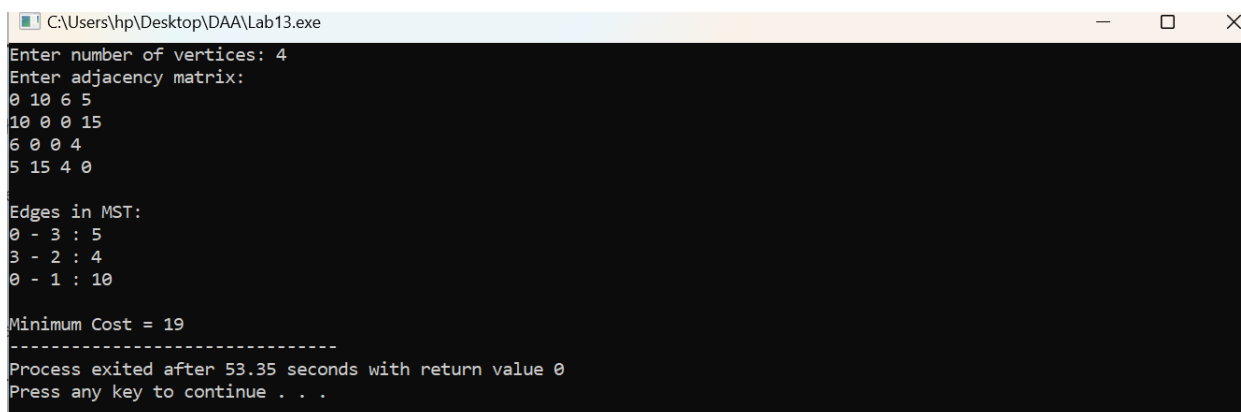
        if (!used[i] && (u == -1 || key[i] < key[u]))
            u = i;
used[u] = 1;
if (parent[u] != -1) {
    cout << parent[u] << " - " << u
        << " : " << g[parent[u]][u] << endl;
    cost += g[parent[u]][u];
}
for (int v = 0; v < n; v++)
    if (!used[v] && g[u][v] < key[v]) {
        key[v] = g[u][v];
        parent[v] = u;
    }
}

cout << "\nMinimum Cost = " << cost;

return 0;
}

```

Output:



```

C:\Users\hp\Desktop\DAA\Lab13.exe
Enter number of vertices: 4
Enter adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0

Edges in MST:
0 - 3 : 5
3 - 2 : 4
0 - 1 : 10

Minimum Cost = 19
-----
Process exited after 53.35 seconds with return value 0
Press any key to continue . . .

```

14. Write a program to find Minimum Spanning tree using Kruskal's algorithm.

```
#include <iostream>

#include <algorithm>

using namespace std;

struct Edge {
    int u, v, weight;
};

bool compare(Edge a, Edge b) {
    return a.weight < b.weight;
}

int parent[100];

int find(int x) {
    if (parent[x] == x)
        return x;
    return parent[x] = find(parent[x]);
}

void unionSet(int a, int b) {
    a = find(a);
    b = find(b);
    if (a != b)
        parent[b] = a;
}

int main() {
    int n, e;
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    Edge edges[e];
```

```

cout << "Enter edges (source destination weight):\n";

for (int i = 0; i < e; i++)
    cin >> edges[i].u >> edges[i].v >> edges[i].weight;

sort(edges, edges + e, compare);

for (int i = 0; i < n; i++)
    parent[i] = i;

int totalCost = 0, count = 0;

cout << "\nEdges in Minimum Spanning Tree:\n";

for (int i = 0; i < e && count < n - 1; i++) {
    int u = edges[i].u;
    int v = edges[i].v;
    if (find(u) != find(v)) {
        cout << u << " - " << v << " : "
            << edges[i].weight << endl;

        totalCost += edges[i].weight;

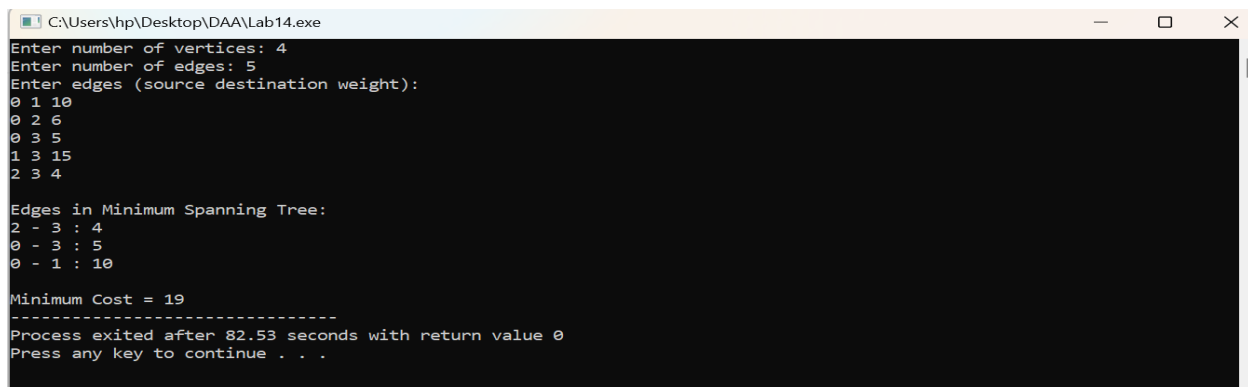
        unionSet(u, v);

        count++;
    } } cout << "\nMinimum Cost = " << totalCost;

return 0;}

```

Output:



```

C:\Users\hp\Desktop\DAA\Lab14.exe
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (source destination weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edges in Minimum Spanning Tree:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10

Minimum Cost = 19
-----
Process exited after 82.53 seconds with return value 0
Press any key to continue . . .

```

15. Write a program to find Single Source Shortest Path using Dijkstra's algorithm

```
#include <iostream>

using namespace std;

#define INF 9999

int main() {
    int n, source;

    cout << "Enter number of vertices: ";

    cin >> n;

    int graph[n][n];

    cout << "Enter adjacency matrix:\n";

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            cin >> graph[i][j];

            if (graph[i][j] == 0 && i != j)
                graph[i][j] = INF;
        }

    cout << "Enter source vertex: ";

    cin >> source;

    int distance[n], visited[n];

    for (int i = 0; i < n; i++) {
        distance[i] = graph[source][i];
        visited[i] = 0;
    } distance[source] = 0;

    for (int count = 0; count < n - 1; count++) {
        int u = -1, minDistance = INF;

        for (int i = 0; i < n; i++)
```

```

if (!visited[i] && distance[i] < minDistance) {
    minDistance = distance[i];
    u = i;
} if (u == -1)
    break;
visited[u] = 1;
for (int v = 0; v < n; v++)
    if (!visited[v] && graph[u][v] != INF &&
        distance[u] + graph[u][v] < distance[v])
        distance[v] = distance[u] + graph[u][v];
} cout << "\nShortest distances from vertex " << source << ":\n";
for (int i = 0; i < n; i++) {
    if (distance[i] == INF)
        cout << source << " -> " << i << " = Not Reachable\n";
    else
        cout << source << " -> " << i << " = " << distance[i] << endl;
}return 0;
}

```

Output:



```

C:\Users\hpl\Desktop\DAA\La x + v
Enter number of vertices: 4
Enter adjacency matrix:
0 10 5 0
0 0 2 1
0 3 0 9
0 0 0 0
Enter source vertex: 0

Shortest distances from vertex 0:
0 -> 0 = 0
0 -> 1 = 8
0 -> 2 = 5
0 -> 3 = 9

-----
Process exited after 46.86 seconds with return value 0
Press any key to continue . . . |

```

16. Write a program to find solution of Knapsack Instance (0/1).

```
#include <iostream>

using namespace std;

int main() {
    int n, capacity;
    cout << "Enter number of items: ";

    cin >> n;

    int weight[n], profit[n];
    cout << "Enter weights: ";

    for (int i = 0; i < n; i++)
        cin >> weight[i];

    cout << "Enter profits: ";

    for (int i = 0; i < n; i++)
        cin >> profit[i];

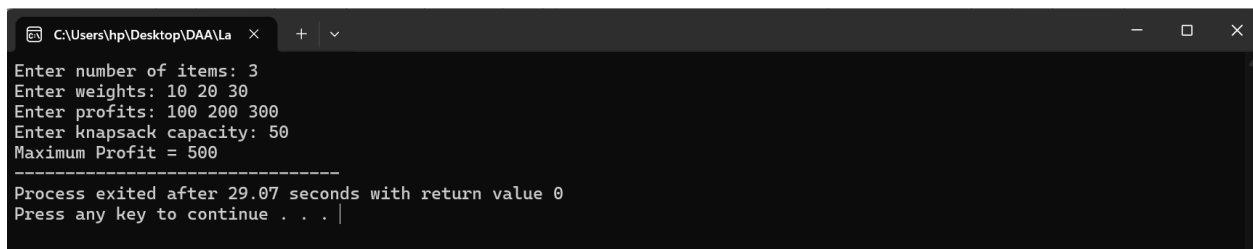
    cout << "Enter knapsack capacity: ";

    cin >> capacity;

    int dp[n + 1][capacity + 1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (weight[i - 1] <= w)
                dp[i][w] = max(profit[i - 1] +
                               dp[i - 1][w - weight[i - 1]],
                               dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }
```

```
}  
  
}  
  
cout << "Maximum Profit = " << dp[n][capacity];  
  
return 0;  
  
}
```

Output:



A screenshot of a terminal window with a dark background. The window title bar shows the file path "C:\Users\hnp\Desktop\DAA\La" and standard window controls. The terminal displays the following text: "Enter number of items: 3", "Enter weights: 10 20 30", "Enter profits: 100 200 300", "Enter knapsack capacity: 50", and "Maximum Profit = 500". A horizontal line of dashes follows. At the bottom, it says "Process exited after 29.07 seconds with return value 0" and "Press any key to continue . . . |".

```
C:\Users\hnp\Desktop\DAA\La x + v  
Enter number of items: 3  
Enter weights: 10 20 30  
Enter profits: 100 200 300  
Enter knapsack capacity: 50  
Maximum Profit = 500  
-----  
Process exited after 29.07 seconds with return value 0  
Press any key to continue . . . |
```

17. Write a program to find solution of Matrix Chain Multiplication.

```
#include <iostream>

using namespace std;

int main() {
    int n;

    cout << "Enter number of matrices: ";

    cin >> n;

    int p[n + 1];

    cout << "Enter dimensions: ";

    for (int i = 0; i <= n; i++)
        cin >> p[i];

    int dp[n + 1][n + 1];

    for (int i = 1; i <= n; i++)
        dp[i][i] = 0;

    for (int len = 2; len <= n; len++) {
        for (int i = 1; i <= n - len + 1; i++) {
            int j = i + len - 1;

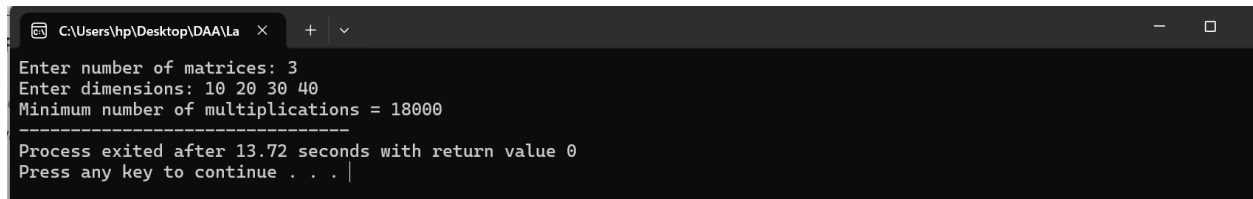
            dp[i][j] = 999999999;

            for (int k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k + 1][j]
                    + p[i - 1] * p[k] * p[j];

                if (cost < dp[i][j])
                    dp[i][j] = cost;
            }
        }
    }
}
```

```
    cout << "Minimum number of multiplications = "  
        << dp[1][n];  
  
    return 0;  
}
```

Output:



```
C:\Users\hnp\Desktop\DAA\La > Enter number of matrices: 3  
Enter dimensions: 10 20 30 40  
Minimum number of multiplications = 18000  
-----  
Process exited after 13.72 seconds with return value 0  
Press any key to continue . . . |
```

18. Write a program to find shortest path using All Pair Shortest Path algorithm.

```
#include <iostream>

using namespace std;

#define INF 9999

int main() {

    int n;

    cout << "Enter number of vertices: ";

    cin >> n;

    int dist[n][n];

    cout << "Enter adjacency matrix:\n";

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            cin >> dist[i][j];

            if (dist[i][j] == 0 && i != j)
                dist[i][j] = INF;
        }

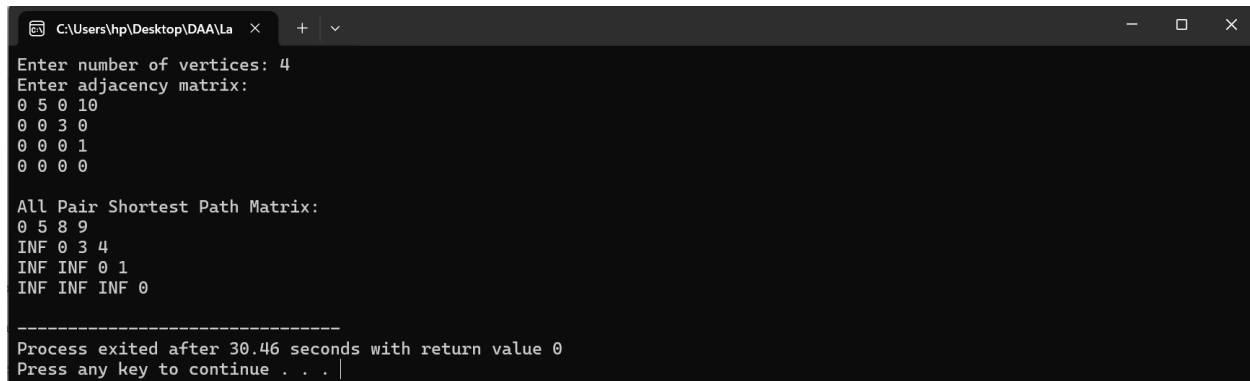
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] != INF && dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];

    cout << "\nAll Pair Shortest Path Matrix:\n";

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF)
```

```
        cout << "INF ";  
    else  
        cout << dist[i][j] << " ";  
    }  
    cout << endl;  
}  
return 0;  
}
```

Output:



```
C:\Users\hp\Desktop\DAA\La x + v  
Enter number of vertices: 4  
Enter adjacency matrix:  
0 5 0 10  
0 0 3 0  
0 0 0 1  
0 0 0 0  
  
All Pair Shortest Path Matrix:  
0 5 8 9  
INF 0 3 4  
INF INF 0 1  
INF INF INF 0  
  
-----  
Process exited after 30.46 seconds with return value 0  
Press any key to continue . . . |
```

19. Write a program to Traverse Graph – Depth First Search, Breadth First Search.

```
#include <iostream>

using namespace std;

int graph[10][10], visited[10], n;

void DFS(int v) {
    cout << v << " ";
    visited[v] = 1;
    for (int i = 0; i < n; i++)
        if (graph[v][i] == 1 && !visited[i])
            DFS(i);
}

void BFS(int start) {
    int queue[10], front = 0, rear = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;
    queue[rear++] = start;
    visited[start] = 1;
    while (front < rear) {
        int v = queue[front++];
        cout << v << " ";
        for (int i = 0; i < n; i++) {
            if (graph[v][i] == 1 && !visited[i]) {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
    }
}
```

```
int main() {  
    int start;  
  
    cout << "Enter number of vertices: ";  
  
    cin >> n;  
  
    cout << "Enter adjacency matrix:\n";  
  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < n; j++)  
            cin >> graph[i][j];  
  
    cout << "Enter starting vertex: ";  
  
    cin >> start;  
  
    for (int i = 0; i < n; i++)  
        visited[i] = 0;  
  
    cout << "\nDFS: ";  
  
    DFS(start);  
  
    cout << "\nBFS: ";  
  
    BFS(start);  
  
    return 0;  
}
```

Output:



```
C:\Users\hnp\Desktop\DAA\La x + v  
Enter number of vertices: 4  
Enter adjacency matrix:  
0 1 1 0  
1 0 0 1  
1 0 0 1  
0 1 1 0  
Enter starting vertex: 0  
  
DFS: 0 1 3 2  
BFS: 0 1 2 3  
-----  
Process exited after 29.68 seconds with return value 0  
Press any key to continue . . . |
```

20. Write a program to find all solutions for N-Queen problem using backtracking.

```
#include <iostream>

using namespace std;

int board[20][20], n;

bool isSafe(int row, int col) {
    for (int i = 0; i < row; i++)
        if (board[i][col])
            return false;

    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return false;

    for (int i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++)
        if (board[i][j])
            return false;

    return true;
}

void solve(int row) {
    if (row == n) {
        cout << "Solution:\n";

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++)
                cout << board[i][j] << " ";

            cout << endl;
        }

        cout << endl;

        return;
    }
}
```

```

    }

    for (int col = 0; col < n; col++) {
        if (isSafe(row, col)) {
            board[row][col] = 1;
            solve(row + 1);
            board[row][col] = 0;
        }
    }
}

int main() {
    cout << "Enter value of N: ";
    cin >> n;

    solve(0);

    return 0;
}

```

Output:

```

C:\Users\hp\Desktop\DAA\La
Enter value of N: 4
Solution:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Solution:
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

-----
Process exited after 8.029 seconds with return value 0
Press any key to continue . . .

```

21. Write a program for Graph Colouring using backtracking.

```
#include <iostream>

using namespace std;

int graph[20][20], color[20], n, m;

bool isSafe(int v, int c) {
    for (int i = 0; i < n; i++)
        if (graph[v][i] && color[i] == c)
            return false;

    return true;
}

bool graphColoring(int v) {
    if (v == n)
        return true;

    for (int c = 1; c <= m; c++) {
        if (isSafe(v, c)) {
            color[v] = c;

            if (graphColoring(v + 1))
                return true;

            color[v] = 0;
        }
    }

    return false;
}

int main() {
    cout << "Enter number of vertices: ";

    cin >> n;
```

```

cout << "Enter adjacency matrix:\n";

for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        cin >> graph[i][j];

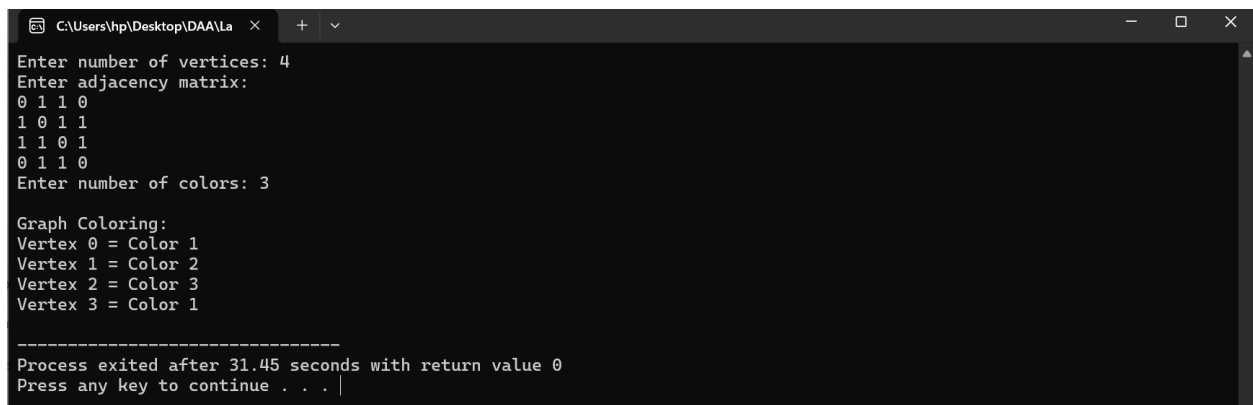
cout << "Enter number of colors: ";
cin >> m;

if (graphColoring(0)) {
    cout << "\nGraph Coloring:\n";
    for (int i = 0; i < n; i++)
        cout << "Vertex " << i << " = Color " << color[i] << endl;
}
else
    cout << "Graph cannot be colored using " << m << " colors.";

return 0;
}

```

Output:



```

C:\Users\hp\Desktop\DAA\La x + v
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 1 1
1 1 0 1
0 1 1 0
Enter number of colors: 3

Graph Coloring:
Vertex 0 = Color 1
Vertex 1 = Color 2
Vertex 2 = Color 3
Vertex 3 = Color 1

-----
Process exited after 31.45 seconds with return value 0
Press any key to continue . . . |

```